ECM2433
Mock Paper 2
The C Family: C, C++, and Rust

Time allowed: 2 Hours

Question 1

(a) A programmer is writing a C function to manage a dynamic list of student IDs:

```c
int *manage_list(int size) {
    int *list = malloc(size * sizeof(int));
    for (int i = 0; i <= size; i++) {
        list[i] = i * 10;
    }
    return list;
}
```

(i) Identify the logic error in the `for` loop and explain the likely runtime consequence.
(ii) If this code is compiled with Address Sanitization (`-fsanitize=address`), how would the output differ compared to a standard execution?
(4 marks)

(b) Explain the difference between `sizeof()` and `strlen()` when applied to the following declaration:
`char hello[20] = "Hello world";`
(4 marks)

(c) In C, how does the `extern "C"` linkage specification affect name mangling, and why is it necessary when calling functions defined in a `.c` file from a C++ program?
(4 marks)

(d) Write a C function `void swap_anything(void *p, void *q, size_t size)` that swaps the contents of two memory locations of any data type.
(8 marks)

(e) A programmer uses `scanf("%s", buffer)` to read a string. Explain why this is dangerous and suggest a better format string to prevent buffer overflow.
(5 marks)

Question 2

(a) Consider the following C++ class hierarchy:

```
class Sensor {
public:
    std::string name;
    Sensor(std::string n) : name(n) {}
    virtual double read() { return 0.0; }
    virtual ~Sensor() { std::cout << "Destructing Sensor"; }
};

class TempSensor : public Sensor {
    double reading;
public:
    TempSensor(std::string n, double r) : Sensor(n), reading(r)
       {}
    double read() override { return reading; }
    ~TempSensor() { std::cout << "Destructing TempSensor"; }
};
```

(i) If a programmer executes `Sensor *s = new TempSensor("T1", 25.0); delete s;`, explain why the output might be incomplete and how to fix it.
(ii) Explain the difference between static and dynamic binding in this context.
(6 marks)

(b) C++ supports function overloading. Explain how the compiler uses "Name Mangling" to support two functions with the same name but different parameter types.
(4 marks)

(c) A developer is using `std::cin >> age >> course;` to read `21` and `"Computer Science"`. Explain why the `course` variable will only contain `"Computer"` and suggest the correct C++ function to read the full line.
(4 marks)

(d) Using the C++ Standard Template Library (STL), write a code snippet that takes a `std::vector<Student>` and sorts it by `mark` (descending), then by `name` (alphabetical) for students with equal marks. Use a lambda expression.
(6 marks)

(e) Define a simple template class `Pair<T>` that holds two values of type `T` and provides a method `bool are_equal()` using operator overloading.
(5 marks)

Question 3

(a) Rust's `Option<T>` and `Result<T, E>` enums are used to handle values that might be missing or errors.
(i) Write a Rust function signature for `find_task` that takes a slice of Strings and a keyword, returning the index of the first match.
(ii) Show how to handle the return value of this function using a `match` statement.
(4 marks)

(b) Explain the concept of **interior mutability** in Rust, specifically referring to the relationship between `Arc<T>` and `Mutex<T>` in multi-threaded programs.
(4 marks)

(c) Consider the following Rust code:

```
fn main() {
    let mut tasks = vec![String::from("Task 1")];
    let r1 = &tasks[0];
    tasks.push(String::from("Task 2"));
    println!("{}", r1);
}
```

Explain why this code fails to compile, referring to Rust's rules on references and mutability.
(6 marks)

(d) Write a Rust expression using iterator methods (`iter`, `filter`, `map`, `collect`) that takes a `Vec<i32>`, keeps only values greater than 10, doubles them, and returns a new `Vec`.
(4 marks)

(e) What is "deref coercion" in Rust, and how does it allow a `Box<Song>` to be passed to a function expecting `&Song`?
(4 marks)

Question 4

(a) Compare error handling in C, C++, and Rust:
(i) Describe the disadvantage of C's return-code approach when a function returns a valid integer that could also be an error code.
(ii) How does Rust's `?` operator improve upon the manual checking of `Result` types?
(6 marks)

(b) In C++, what are **Smart Pointers**? Contrast `std::unique_ptr` and `std::shared_ptr`, explaining how they relate to the concept of ownership.
(6 marks)

(c) You are tasked with parallelizing a task in Rust (e.g., a Caesar Cipher or word count).
(i) Explain why you must use a `move` closure when spawning a thread with `std::thread::spawn`.
(ii) Describe how an `mpsc` channel can be used to collect results from multiple worker threads.
(8 marks)

(d) In C++, operator overloading allows custom classes to behave like built-in types. Give the prototype for overloading the `+` operator to add an integer (seconds) to a custom `MyTime`.
(5 marks)

End of Paper